

Week 6 Metric Formula Cheat Sheet

Model Evaluation Metrics and Experiment Tracking

AI Engineering Academy, Gamut Technology Services. Learner reference. All worked numbers verified against the cohort's pinned libraries: scikit-learn 1.8.0, sacrebleu 2.6.0, rouge-score 0.1.2, nltk 3.10.1.

How to use this sheet: each metric gets a definition, the formula, a step-by-step calculation recipe, and a fully worked example. The classification metrics share one running scenario so you can trace the same four counts through every formula. The generated-text metrics share one hypothesis and reference pair for the same reason.

1. Confusion Matrix Foundations

Scenario (used for sections 1 to 8): a Cordwell Home and Hardware review classifier flags product reviews that report a safety defect. Test set: 100 reviews, 12 actually report a defect (positive class), 88 do not.

Every count below comes from comparing the model's prediction to the true label:

- TP (true positive): predicted defect, actually a defect. Here TP = 8.
- FP (false positive): predicted defect, actually not. Here FP = 2.
- FN (false negative): predicted not a defect, actually a defect. Here FN = 4.
- TN (true negative): predicted not a defect, actually not. Here TN = 86.

	Predicted: no defect	Predicted: defect
Actual: no defect	TN = 86	FP = 2
Actual: defect	FN = 4	TP = 8

Library check: `sklearn.metrics.confusion_matrix(y_true, y_pred)` returns rows as actual and columns as predicted, ordered by sorted label. For binary labels 0 and 1 that is `[[TN, FP], [FN, TP]]`. Verified output for this scenario: `[[86, 2], [4, 8]]`.

2. Accuracy

Fraction of all predictions that are correct, regardless of class.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

How to calculate:

1. Add the correct predictions: TP + TN.
2. Divide by the total number of predictions.

Worked: $(8 + 86) / 100 = 0.9400$. Verified: `accuracy_score(y_true, y_pred)` returns 0.9400.

The imbalance trap: a model that predicts no defect for every review scores 0.88 accuracy on this set while catching zero defects (verified). On imbalanced data, always compare accuracy to the majority-class baseline and lean on precision, recall, and F1.

3. Precision

Of everything the model flagged as positive, how much really was. High precision means few false alarms.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

How to calculate:

1. Count all positive predictions: TP + FP.

2. Divide the true positives by that count.

Worked: $8 / (8 + 2) = 0.8000$. Of the 10 reviews flagged as defect reports, 8 were real.

4. Recall

Of everything that really was positive, how much the model caught. High recall means few misses.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

How to calculate:

1. Count all actual positives: $\text{TP} + \text{FN}$.
2. Divide the true positives by that count.

Worked: $8 / (8 + 4) = 0.6667$. Of the 12 real defect reports, the model caught 8 and missed 4. For a safety use case, those 4 misses are the number that matters.

5. F1 Score

Harmonic mean of precision and recall. The harmonic mean punishes imbalance: a model cannot buy a good F1 by maxing one side while tanking the other.

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

$$\text{Equivalently: } \text{F1} = 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN})$$

How to calculate:

1. Compute precision and recall first.
2. Multiply them, double the result, then divide by their sum.

Worked: $2 \times (0.8000 \times 0.6667) / (0.8000 + 0.6667) = 0.7273$. Verified: `f1_score(y_true, y_pred)` returns 0.7273. Note the gap: accuracy said 0.94, F1 says 0.73. F1 is telling the truth about the minority class.

6. Balanced Accuracy

Mean of the per-class recalls. Fixes the accuracy imbalance trap by giving each class an equal vote regardless of its size. For binary problems this is the mean of sensitivity (positive recall, TPR) and specificity (negative recall, TNR).

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN}) \quad \text{TNR} = \text{TN} / (\text{TN} + \text{FP})$$

$$\text{Balanced Accuracy} = (\text{TPR} + \text{TNR}) / 2$$

How to calculate:

1. Compute recall for the positive class: $\text{TP} / (\text{TP} + \text{FN})$.
2. Compute recall for the negative class: $\text{TN} / (\text{TN} + \text{FP})$.
3. Average them. For multiclass, average the recall of every class.

Worked: $\text{TPR} = 8/12 = 0.6667$, $\text{TNR} = 86/88 = 0.9773$, $\text{BA} = (0.6667 + 0.9773) / 2 = 0.8220$. Verified: `balanced_accuracy_score(y_true, y_pred)` returns 0.8220.

The honest baseline: the all-negative model that scored 0.88 raw accuracy scores exactly 0.5 balanced accuracy (verified), the same as random guessing. Any constant-prediction strategy lands at $1 / n_{\text{classes}}$ worth of recall averaged out, so 0.5 is the binary floor to beat, not 0.88.

7. Matthews Correlation Coefficient (MCC)

Pearson correlation between predicted and true labels, computed straight from the confusion matrix. Uses all four cells, so it only rewards a model that is doing well on both classes. Range is -1 to +1: +1 is perfect, 0 is no better than chance, -1 is perfect disagreement.

$$\text{MCC} = (\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) /$$

$$\sqrt{(TP+FP) \times (TP+FN) \times (TN+FP) \times (TN+FN)}$$

How to calculate:

1. Numerator: TP x TN minus FP x FN.
2. Denominator: multiply the four marginal totals (predicted positives, actual positives, predicted negatives as TN+FN, actual negatives as TN+FP), then take the square root.
3. Divide. If any marginal is zero the denominator is zero; sklearn returns 0 in that case.

Worked: numerator = $8 \times 86 - 2 \times 4 = 680$; denominator = $\sqrt{10 \times 12 \times 88 \times 90} = 974.88$; MCC = $680 / 974.88 = 0.6975$. Verified: `matthews_corrcoef(y_true, y_pred)` returns 0.6975. The all-negative baseline scores exactly 0.0 (verified): MCC cannot be gamed by predicting the majority class.

When to reach for it: single-number summary on imbalanced data. Unlike F1 it also counts the true negatives, so it will drop when the negative class degrades even while precision and recall on the positive class look stable.

8. Cohen's Kappa

Agreement between predictions and truth, corrected for the agreement you would expect by chance given each side's label frequencies. Originally an inter-annotator agreement statistic, which is exactly how Week 6 reuses it for human-in-the-loop and LLM-as-a-judge label audits: swap `y_pred` for a second rater.

```
p_o = observed agreement = accuracy
p_e = sum over classes of P(pred = c) x P(true = c)
kappa = (p_o - p_e) / (1 - p_e)
```

How to calculate:

1. Observed agreement `p_o` is plain accuracy: 0.9400 here.
2. Chance agreement `p_e` from the marginals: the model predicts positive 10% of the time and 12% of labels are positive, so chance positive agreement is 0.10×0.12 ; negative side is 0.90×0.88 . `p_e` = $0.012 + 0.792 = 0.8040$.
3. Kappa rescales the headroom above chance: how much of the possible improvement over `p_e` was achieved.

Worked: `kappa` = $(0.9400 - 0.8040) / (1 - 0.8040) = 0.1360 / 0.1960 = 0.6939$. Verified: `cohen_kappa_score(y_true, y_pred)` returns 0.6939. The all-negative baseline scores 0.0 (verified), because 0.88 accuracy is exactly its chance agreement.

Reading the number: common rule-of-thumb bands (0.41 to 0.60 moderate, 0.61 to 0.80 substantial, above 0.80 near-perfect) are conventions from Landis and Koch, not statistics; state the bands you use. Note `kappa` 0.6939 and MCC 0.6975 land close here, which is typical: for binary problems they often track each other, with `kappa` the standard choice for rater agreement and MCC for classifier quality.

9. Multiclass Averaging: Macro, Micro, Weighted

With more than two classes, sklearn computes precision, recall, and F1 per class (one-vs-rest), then combines them. The averaging choice changes the story.

```
Macro F1    = mean of per-class F1 scores (every class counts equally)
Micro F1    = F1 computed from pooled TP, FP, FN across all classes
Weighted F1 = sum over classes of (support_c / N) x F1_c
```

Worked example: 10 predictions over 3 ticket categories with supports 6, 3, and 1. Verified per-class results:

Class	Support	Precision	Recall	F1
0	6	0.7143	0.8333	0.7692
1	3	0.6667	0.6667	0.6667

Class	Support	Precision	Recall	F1
2	1	0.0000	0.0000	0.0000

- Macro = $(0.7692 + 0.6667 + 0.0000) / 3 = 0.4786$. The empty class 2 drags it hard: macro exposes weak minority classes.
- Micro = 0.7000. In single-label multiclass classification, micro F1 equals accuracy (verified: both 0.7000 here).
- Weighted = $(6 \times 0.7692 + 3 \times 0.6667 + 1 \times 0.0000) / 10 = 0.6615$. Support-weighted, so big classes dominate.

Rule of thumb: report macro when minority classes matter, weighted when you want an honest overall number, and treat micro as accuracy in disguise.

10. Generated-Text Metrics: Shared Worked Example

Sections 10a to 10e all score the same pair, a Cordwell order-status summary against its reference:

```
reference: "the delivery driver left the package at the front door"    (10 tokens)
hypothesis: "the driver left the package at the door"                (8 tokens)
```

Tokenization for the hand calculations is whitespace split, matching the verification scripts. Production scoring should use each library's default tokenizer and report it, because scores are only comparable under a pinned configuration.

10a. BLEU (sacrebleu 2.6.0)

Precision-oriented: how much of the hypothesis appears in the reference, across n-gram sizes 1 to 4, discounted for being too short.

```
p_n = clipped n-gram matches / total n-grams in hypothesis    (n = 1..4)
BP  = 1                                                         if sys_len > ref_len
BP  = exp(1 - ref_len / sys_len)   otherwise
BLEU = BP x exp( (1/4) x sum(log p_n) ) x 100
```

How to calculate:

1. For each n from 1 to 4, count hypothesis n-grams that also appear in the reference, clipping each n-gram's count at its reference count.
2. Divide by the total n-grams in the hypothesis to get p_1 through p_4.
3. Compute the brevity penalty from hypothesis and reference lengths.
4. Take the geometric mean of the four precisions, multiply by BP, scale to 0 to 100.

Worked (verified): precisions [100.0, 71.4286, 66.6667, 60.0], sys_len 8, ref_len 10, BP = $\exp(1 - 10/8) = 0.7788$, BLEU = 56.94.

```
import sacrebleu
bleu = sacrebleu.corpus_bleu([hypothesis], [[reference]]) # hyps first, then list of reference lists
bleu.score, bleu.precisions, bleu.bp # 56.9389, [100.0, 71.43, 66.67, 60.0], 0.7788
```

- sacrebleu reports on a 0 to 100 scale, not 0 to 1. A 0.57 and a 57 are the same result from different tools.
- Always report the brevity penalty with the score. A high BLEU with BP well below 1 means the model is being punished for short output, and a BLEU compared across papers without matching tokenizer settings is meaningless.
- Argument order: hypotheses first, then references as a list of reference lists. This is the reverse of rouge-score below.

10b. ROUGE-1 and ROUGE-2 (rouge-score 0.1.2)

Recall-oriented by design, though the library reports precision, recall, and F. ROUGE-N counts n-gram overlap between hypothesis and reference.

```
ROUGE-N recall    = overlapping n-grams / n-grams in reference
ROUGE-N precision = overlapping n-grams / n-grams in hypothesis
ROUGE-N F         = 2PR / (P + R)
```

How to calculate (ROUGE-1):

1. Count unigram overlap with per-token clipping: 8 shared tokens here (the x3, driver, left, package, at, door).
2. Recall = 8/10 = 0.8000, precision = 8/8 = 1.0000, F = 0.8889.

Worked (verified): rouge1 P=1.0000 R=0.8000 F=0.8889; rouge2 P=0.7143 R=0.5556 F=0.6250 (bigrams: 5 overlapping, 7 in hypothesis, 9 in reference).

```
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=False)
scores = scorer.score(reference, hypothesis) # TARGET FIRST, prediction second
```

- **Silent-error warning:** `scorer.score(target, prediction)` takes the reference first. Swapping the arguments swaps precision and recall with no error raised, and it is the reverse of sacrebleu's order.

10c. ROUGE-L (rouge-score 0.1.2)

Replaces n-gram counts with the longest common subsequence (LCS): the longest sequence of tokens appearing in both texts in the same order, not necessarily contiguously. Rewards preserved ordering without requiring exact adjacency.

```
R_lcs = LCS(ref, hyp) / len(ref)
P_lcs = LCS(ref, hyp) / len(hyp)
ROUGE-L F = 2 x P_lcs x R_lcs / (P_lcs + R_lcs)
```

How to calculate:

1. Find the LCS length. Here every hypothesis token appears in the reference in order, so LCS = 8.
2. Recall = 8/10 = 0.8000, precision = 8/8 = 1.0000, F = 0.8889.

Worked (verified): rougeL P=1.0000 R=0.8000 F=0.8889. It matches ROUGE-1 here only because the shared tokens happen to be in order; a shuffled hypothesis would keep ROUGE-1 and drop ROUGE-L.

10d. METEOR (nltk 3.10.1)

Aligns hypothesis to reference using exact matches plus stems and WordNet synonyms, weights recall 9x over precision, then penalizes fragmented word order via contiguous chunks.

```
P = matches / len(hyp)          R = matches / len(ref)
Fmean  = 10PR / (R + 9P)
Penalty = 0.5 x (chunks / matches)^3
METEOR  = Fmean x (1 - Penalty)
```

How to calculate:

1. Align tokens (exact, then stem, then synonym). Here: 8 matches.
2. P = 8/8 = 1.0, R = 8/10 = 0.8, Fmean = 10(0.8)/(0.8 + 9) = 0.8163.
3. Group matches into contiguous aligned chunks: 3 chunks here, penalty = 0.5 x (3/8)^3 = 0.0264.
4. METEOR = 0.8163 x (1 - 0.0264) = 0.7948.

Worked (verified): `meteor_score([reference.split()], hypothesis.split()) = 0.7948`. Both arguments are token lists, references wrapped in an outer list. Requires `nltk.download("wordnet")` once per machine.

10e. BERTScore (bert-score 0.3.13)

Embeds every token in both texts with a pretrained transformer, then matches on cosine similarity instead of surface overlap, so paraphrases score well.

```
R_BERT = (1/len(ref)) x sum over ref tokens of max cosine sim to any hyp token
P_BERT = (1/len(hyp)) x sum over hyp tokens of max cosine sim to any ref token
F_BERT = 2 x P_BERT x R_BERT / (P_BERT + R_BERT)
```

How to calculate:

1. Embed both token sequences with the scoring model (contextual embeddings).
2. For recall, greedily match each reference token to its most similar hypothesis token; for precision, the reverse.
3. Average the max similarities on each side, then take the harmonic mean.
4. With `rescale_with_baseline=True`, scores are linearly rescaled against a language baseline so typical values spread over a readable range instead of clustering near 1.

Reminder from the deck: all of these surface and embedding metrics can misrank fluent LLM output against human preference (Goyal et al. 2022). They are regression tests, not judges of quality. Pair them with human or LLM-as-a-judge evaluation for anything user-facing.

11. Quick Reference

Metric	Range	Library call	Watch out for
Accuracy	0 to 1	<code>accuracy_score(y_true, y_pred)</code>	Lies on imbalanced data; compare to majority baseline
Precision	0 to 1	<code>precision_score(y_true, y_pred)</code>	Undefined when no positive predictions; set <code>zero_division</code>
Recall	0 to 1	<code>recall_score(y_true, y_pred)</code>	Trivially 1.0 if you predict positive for everything
F1	0 to 1	<code>f1_score(y_true, y_pred)</code>	Pick <code>average=</code> deliberately for multiclass
Macro F1	0 to 1	<code>f1_score(..., average='macro')</code>	Sensitive to tiny classes, by design
Micro F1	0 to 1	<code>f1_score(..., average='micro')</code>	Equals accuracy in single-label multiclass
Weighted F1	0 to 1	<code>f1_score(..., average='weighted')</code>	Big classes can hide minority failure
Balanced Acc.	0 to 1	<code>balanced_accuracy_score(y_true, y_pred)</code>	Constant prediction floors at $1 / n_{\text{classes}}$
MCC	-1 to +1	<code>matthews_corrcoef(y_true, y_pred)</code>	0 when any marginal is empty; 0 for majority-only
Cohen's kappa	-1 to +1	<code>cohen_kappa_score(y_true, y_pred)</code>	Band labels are convention, not statistics
BLEU	0 to 100	<code>sacrebleu.corpus_bleu(hyps, [refs])</code>	Hypotheses first; report BP; pin tokenizer
ROUGE-1, 2, L	0 to 1	<code>RougeScorer(...).score(target, pred)</code>	Target first; swap is silent
METEOR	0 to 1	<code>meteor_score([ref_tokens], hyp_tokens)</code>	Token lists in, needs wordnet data
BERTScore	roughly 0 to 1	<code>bert_score.score(cands, refs, lang='en')</code>	Needs model download; state rescaling and model

Comparability rule: a score is only meaningful next to another score computed with the same library version, tokenizer, and configuration. Log all three in the experiment tracker with every run, and never report a delta without a confidence interval or significance test.

Verified against: *scikit-learn 1.8.0, sacrebleu 2.6.0, rouge-score 0.1.2, nltk 3.10.1. BERTScore section is formula-only (bert-score 0.3.13).*